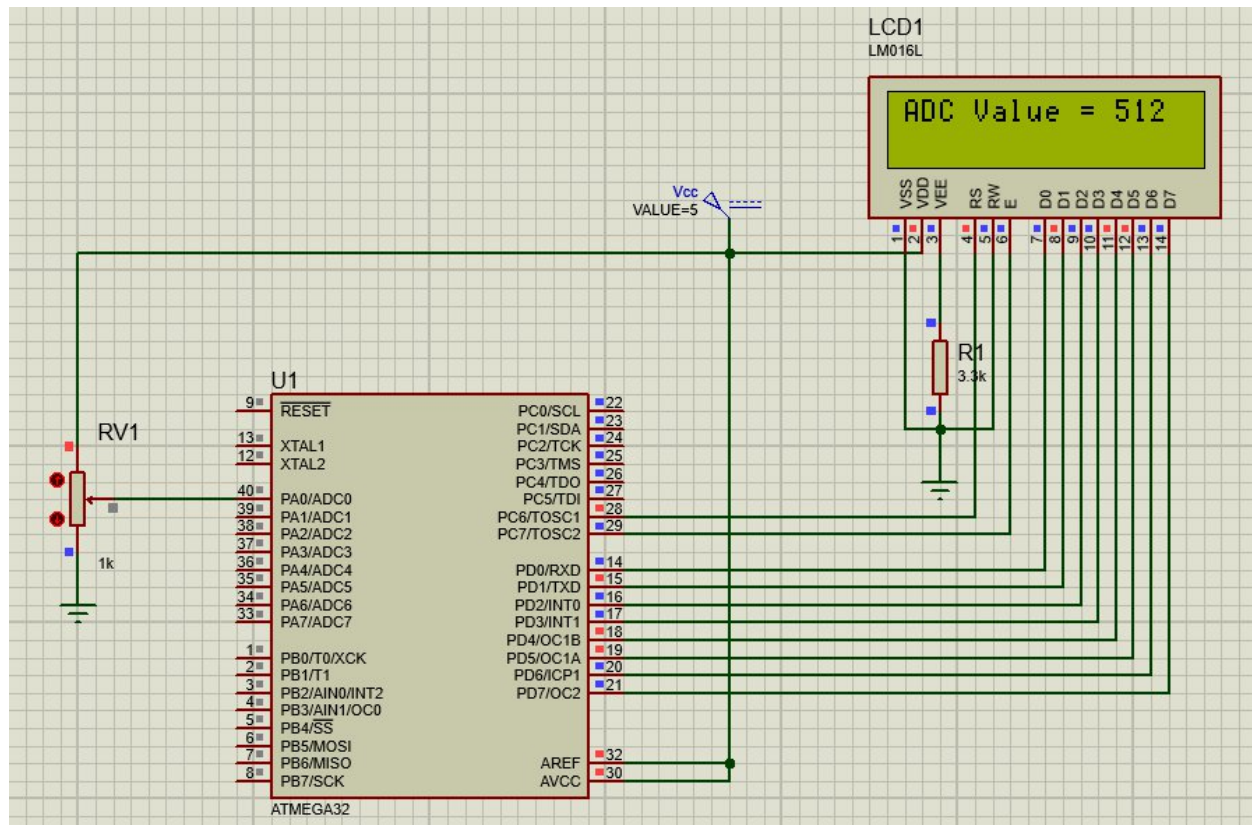


ADC conversion and Display the value in LCD



```

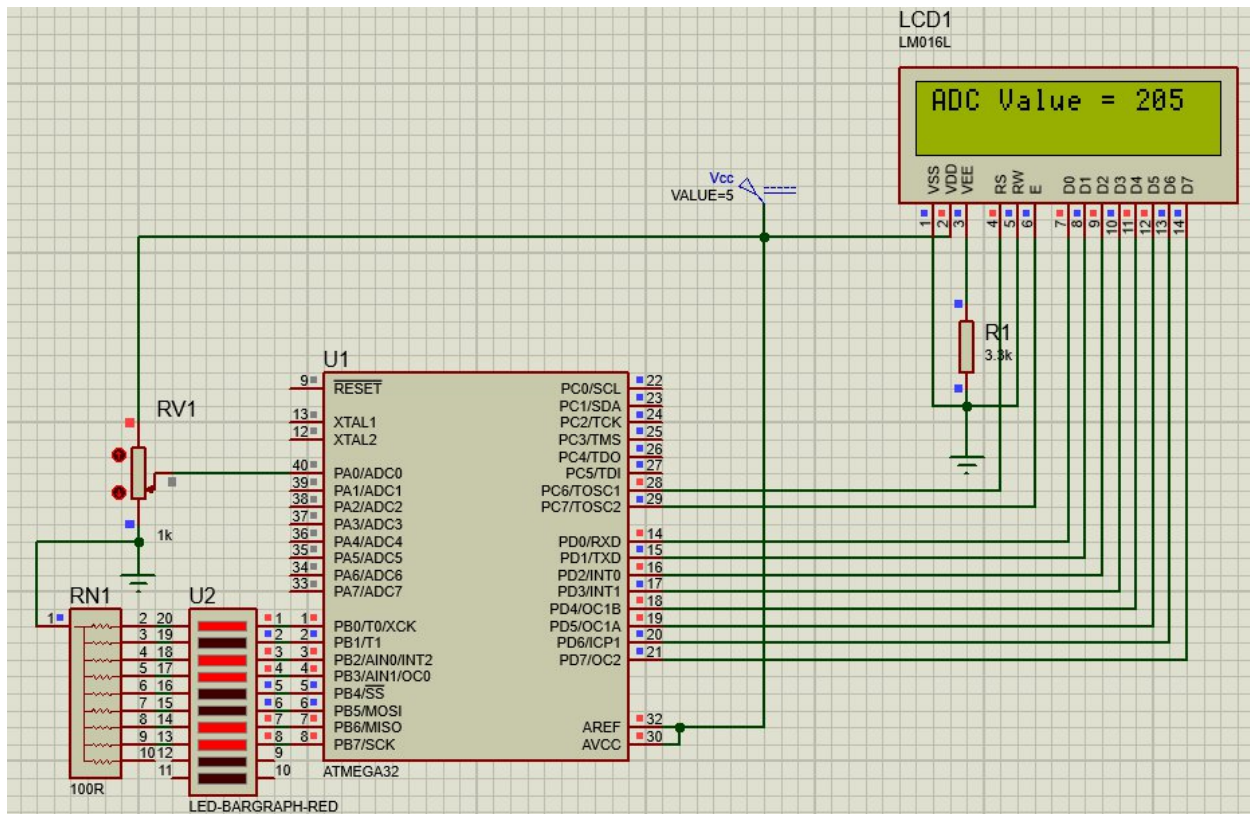
1  #define F_CPU 8000000UL
2  #include <avr/io.h>
3  #include <stdint.h>
4  #include <stdlib.h>
5  #include <util/delay.h>
6  #define D0 eS_PORTD0
7  #define D1 eS_PORTD1
8  #define D2 eS_PORTD2
9  #define D3 eS_PORTD3
10 #define D4 eS_PORTD4
11 #define D5 eS_PORTD5
12 #define D6 eS_PORTD6
13 #define D7 eS_PORTD7
14 #define RS eS_PORTC6
15 #define EN eS_PORTC7
16 #include "lcd.h"
17
18 void adc_init(){
19     ADMUX = (1<<REFS0); // AREF = A Vcc
20     ADCSRA = (1<<ADEN)|(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0); // ADC Enable and prescaler of 128
21 }
22
23 uint16_t ReadADC(uint8_t ch){
24     ch &= 0b00000111;
25     ADMUX &= 0b11100000;
26     ADMUX |= ch;
27     ADCSRA |= (1<<ADSC); // Single start conversion
28     while (!(ADCSRA & (1<<ADIF))); // Wait for conversion
29     // ADCSRA |= (1<<ADIF);
30     return (ADC);
31 }
32
33 int main(void){
34     DDRD = 0xFF;
35     DDRC = 0xFF;
36     Lcd8_Init();
37     adc_init();
38     uint8_t ch = 0;
39     uint16_t getVal;
40     char strVal[4];
41     while(1){
42         getVal = ReadADC(ch);
43         itoa(getVal, strVal, 10);
44         Lcd8_Clear();
45         Lcd8_Set_Cursor(1,0);
46         Lcd8_Write_String("ADC Value = ");
47         Lcd8_Write_String(strVal);
48         _delay_ms(1000);
49     }
50 }

```

Task 1

Display 10-bit ADC value on the LCD screen under following conditions.

- If (adc_result $\leq$ 255), value display in binary representation on eight LED connected to PORTB.
- If (adc_result > 255), Switch on the eight LED connected to PORTB and blink all to represent the PORTB is over flowed.



```

33 int main(void){
34     DDRB = 0xFF;
35     DDRD = 0xFF;
36     DDRC = 0xFF;
37     Lcd8_Init();
38     adc_init();
39     uint8_t ch = 0;
40     uint16_t getVal;
41     char strVal[4];
42     while(1){
43         getVal = ReadADC(ch);
44         itoa(getVal, strVal, 10);
45         Lcd8_Clear();
46         Lcd8_Set_Cursor(1,0);
47         Lcd8_Write_String("ADC Value = ");
48         Lcd8_Write_String(strVal);
49         if (getVal<=255){
50             PORTB = getVal;
51             _delay_ms(1000);
52         }
53         else{
54             PORTB = 0xFF;
55             _delay_ms(200);
56             PORTB = 0x00;
57             _delay_ms(200);
58         }
59     }
60 }

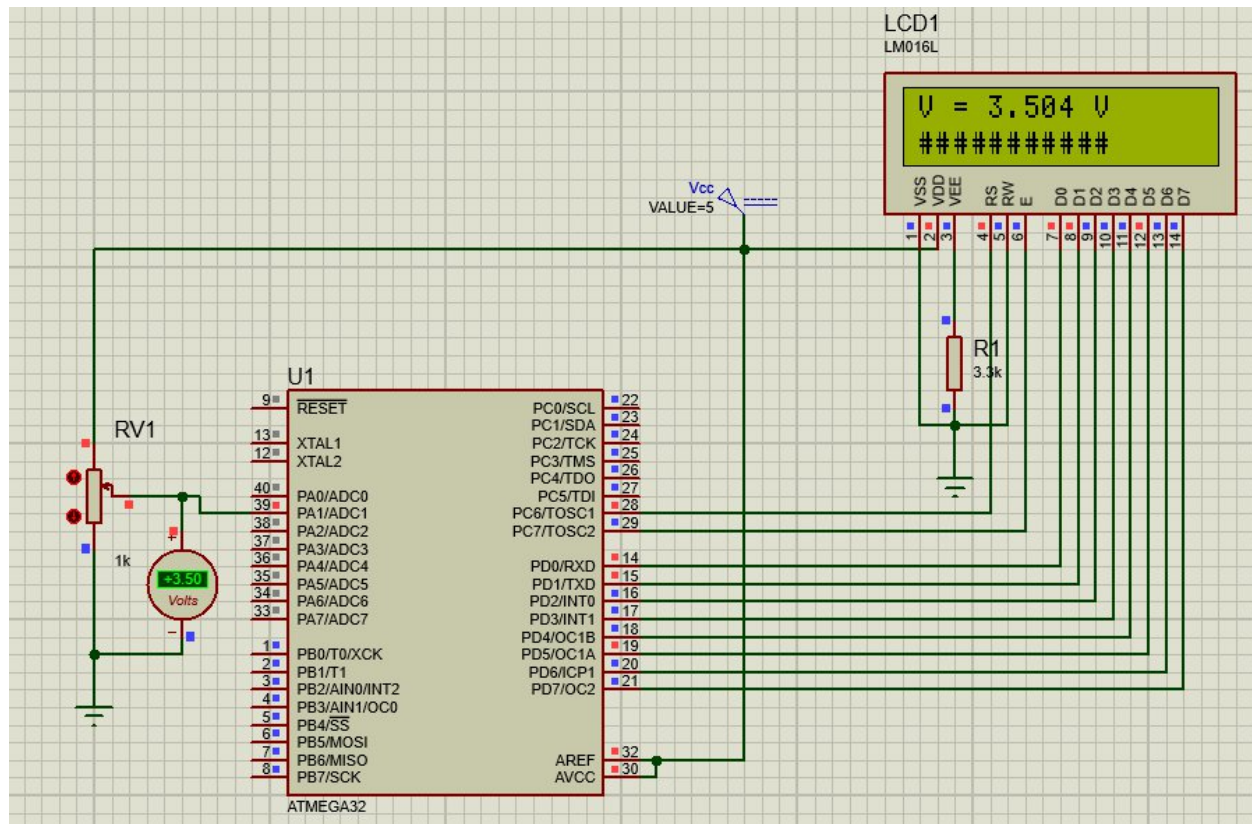
```

Task 2: Voltage measurement & bar graph

Convert ADC reading to **millivolts** and show numeric voltage and a 16-column bar on second LCD row. Circuit: same task 01 + potentiometer or known voltage divider.

Steps

1. Read ADC using channel 01. (ADC1)
2. Compute mV using integer arithmetic to avoid float.
3. Print voltage as x.xxx V (or xxxx mV) on line 1.
4. On line 2, print a bar of up to 16 characters (* or #) proportional to ADC value.



```

18 void adc_init(){
19     ADMUX = (1<<REFS0)|(1<<MUX0); // AREF = A Vcc
20     ADCSRA = (1<<ADEN)|(1<<ADPS2)|(1<<ADPS1)|(1<<ADPS0); // ADC Enable and prescaler of 128
21 }
22
23 uint16_t ReadADC(uint8_t ch){
24     ch &= 0b00000111;
25     ADMUX &= 0b11100001;
26     ADMUX |= ch;
27     ADCSRA |= (1<<ADSC); // Single start conversion
28     while (!(ADCSRA & (1<<ADIF))); // Wait for conversion
29     // ADCSRA |= (1<<ADIF);
30     return (ADC);
31 }
32

```

```

33 int main(void){
34     DDRB = 0xFF;
35     DDRD = 0xFF;
36     DDRC = 0xFF;
37     Lcd8_Init();
38     adc_init();
39     uint8_t ch = 0;
40     uint16_t getVal;
41     char strVal[4];
42     while(1){
43         getVal = ReadADC(ch);
44         int mV = getVal*5.0/1023.0;
45         int mVd = getVal*5%1023;
46         int d = mVd*1000.0/1023.0;
47         int n = getVal*16/1023;
48         itoa(mV, strVal, 10);
49         Lcd8_Clear();
50         Lcd8_Set_Cursor(1,0);
51         Lcd8_Write_String("V = ");
52         Lcd8_Write_String(strVal);
53         itoa(d, strVal, 10);
54         Lcd8_Write_Char('.');
55         while(1){
56             if (mVd<10){
57                 Lcd8_Write_String("00");
58                 Lcd8_Write_String(strVal);
59                 break;
60             }
61             if (mVd<100){
62                 Lcd8_Write_String("0");
63                 Lcd8_Write_String(strVal);
64                 break;
65             }
66             if (mVd<1000){
67                 Lcd8_Write_String(strVal);
68                 break;
69             }
70             else{
71                 Lcd8_Write_String(strVal);
72                 break;
73             }}
74         Lcd8_Write_String(" V");
75         Lcd8_Set_Cursor(2,0);
76         for (int i=0;i<n;i++){
77             Lcd8_Write_Char('#');
78         }
79         _delay_ms(1000);
80     }
81 }

```

Task 3: Temperature monitor (LM35) with threshold alarm

Use ADC to read LM35 on ADC1 (PA1), compute temperature in °C, display and turn on a buzzer/LED if temperature > threshold. Also allow the user to change threshold using potentiometer on ADC2.

Parts: LM35 ($V_{out} = 10 \text{ mV}/^{\circ}\text{C}$), small buzzer on LED, 10k pot for threshold, LCD Display.

Steps

1. Wire LM35 output à PA1
2. Variable resistor (POT) à PA2 to set threshold (0-5V maps to 0-100°C scale, but you can clamp)
3. Read ADC1 for temperature, ADC2 for threshold temperature position.
4. Convert ADC1 à mV à Temp °C
5. Convert ADC2 à threshold temperature °C
6. If Temp > threshold °C, turn on LED/Buzzer and display ALERT on LCD.

```
int main(void){
    DDRB = 0xFF;
    DDRD = 0xFF;
    DDRC = 0xFF;
    Lcd8_Init();
    adc_init();
    int tempC;
    int threshC;
    uint16_t temp_adc;
    uint16_t thresh_adc;
    char strVal[4];
    while(1){
        temp_adc = ReadADC(1); // LM35 connected with ADC1
        thresh_adc = ReadADC(2); // POT connected with ADC2
        tempC = temp_adc*5000UL/1023/10; // 10 mV/C
        threshC = thresh_adc*100UL/1023; // Mapped to 0-100 C
        itoa(tempC, strVal, 10);
        Lcd8_Clear();
        Lcd8_Set_Cursor(1,0);
        Lcd8_Write_String("Temp=");
        Lcd8_Write_String(strVal);
        Lcd8_Write_String("C");
        itoa(threshC, strVal, 10);
        Lcd8_Write_String("Set= ");
        Lcd8_Write_String(strVal);
        Lcd8_Write_String("C");
        if (tempC>threshC)
        {
            Lcd8_Set_Cursor(2,4);
            Lcd8_Write_String("ALERT");
            PORTC |= (1<<PC0); // Set LED ON
        }
        else
        {
            PORTC &= (0<<PC0); // Set LED OFF
        }
        _delay_ms(500);
    }
}
```